

MURAL-Presenter: Multi-Agent Unified Revision-Aware Authoring for Long-Horizon Presentations

Anonymous authors
Paper under double-blind review

Abstract

Automated presentation authoring is a long-horizon multimodal task: individual slides must be informative and well composed, while facts, narrative commitments, and design decisions must remain coherent across the deck and through later edits. Existing Agent systems face a responsibility-granularity trade-off. A single trajectory preserves common context but accumulates heterogeneous execution history; stage- or slide-level decomposition shortens individual trajectories but leaves cross-boundary relations to state handoffs and joint verification. We present MURAL-Presenter (MURAL), a Skill-driven multi-Agent framework that externalizes the full HTML-presentation lifecycle as a reusable Authoring Skill. An Orchestrator maintains a shared deck blueprint and compiles narratively or visually dependent pages into slide groups. Each Group Agent jointly authors and inspects its related pages; whole-deck Review restores global closure after assembly; and a revision router resumes execution at page, group, evidence, or deck scope according to impact. We also introduce THREAD-Bench (Tracking Holistic Requirements and End-to-End Alignment in Decks) for long-horizon consistency, complemented by Present-Bench and SlidesGen-Bench for general generation and DECKBench for multi-turn revision. Working draft: formal E1–E7 results are pending, and no effectiveness claim is made in this version.

1 Introduction

Presentation authoring begins by defining the message, audience, and intended takeaway. Authors then ground materials, choose narrative and design directions, plan slides, review the assembled deck, and revise it after delivery. Prior work likewise identifies noncontiguous evidence selection, outline planning, and render-based revision as consequential stages (Maheshwari et al., 2024; Zheng et al., 2025; Xu et al., 2025). A presentation is therefore a decision lifecycle, not a one-shot page assembly.

The lifecycle is long-horizon because early audience, factual, narrative, and visual decisions may be consumed many slides or revisions later. Local edits should propagate only where needed. Existing evaluations report omitted requirements, unsupported claims, numerical errors, and cross-slide terminology or emphasis mismatches (Chen et al., 2026). We therefore define horizon as the distance over which state remains valid across stages, slides, and versions—not runtime or slide count alone. HTML keeps text, graphics, media, and layout addressable before image, PDF, or PPTX export; CSS, animation, font, and PPTX-structure losses remain explicit delivery limits.

The tension is not Agent count, but whether execution boundaries match deck dependencies. A single Agent preserves one context, yet its materials, plans, observations, and repairs accumulate; growing histories can impair earlier-information use before the nominal limit (Sun et al., 2025). Stage and per-slide decomposition shorten trajectories or enable parallelism, but factual reuse, narrative setup, visual encodings, and opening-closing relations often cross those boundaries. We do not claim inevitable forgetting or drift: decomposition

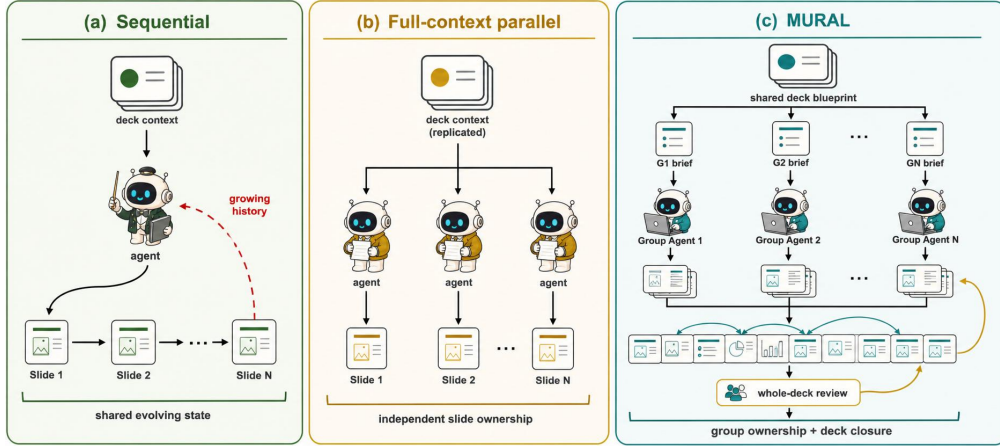


Figure 1: Three page-production topologies and their responsibility boundaries. Sequential generation carries state through one growing history. Full-context parallel copies the same deck context to workers that independently own one slide. MURAL externalizes a shared deck blueprint, compiles slides by cross-page dependencies, and assigns each group to one Agent for joint production and inspection. The distinction is not whether outputs are decomposed or eventually assembled, but where joint responsibility lies: MURAL closes within-group relations before merging and uses whole-deck Review to validate cross-group relations afterward.

helps only when ownership follows dependencies. Controlled studies likewise identify decomposability, information sharing, and validation placement—not Agent count alone—as decisive (Kim et al., 2025).

Who, then, should own a cross-slide relation? Replicated deck context does not create joint ownership, and fixed windows miss noncontiguous bookends, dividers, comparisons, or asset series. MURAL introduces complete, non-overlapping slide groups compiled from narrative purpose, design DNA, assets, and explicit dependencies. A group shares decisions and inspection responsibility, not one template. This dependency-aligned responsibility topology gives relations a bounded owner, leaves independent groups parallel, and makes groups revision units. Multi-Agent execution realizes the topology; it is not the novelty by itself.

The MURAL Authoring Skill makes this lifecycle executable. Its Orchestrator resolves constraints, invokes Material and Research as needed, and writes the storyline, design system, page map, briefs, assets, and groups. A centralized Image stage resolves bitmaps; each Group Agent runs joint render-inspect-revise; whole-deck Review then checks cross-group relations. Figure 1 contrasts sequential, full-context per-slide, and MURAL group-parallel production. The Skill specifies roles, artifacts, quality gates, and continuation paths rather than serving as a prompt template.

Persistent state also supports later editing. Local requests receive targeted patches; within-group changes reactivate that Group Agent; new evidence invokes Research or Image; structural changes replan and recheck affected regions. Add, delete, and reorder operations are therefore structural revisions. Generation-time inspect-diagnose-patch-re-render may transfer without revision-only training, but this remains a hypothesis tested through success, instruction following, untouched-slide preservation, and regressions—not artifact editability alone.

QueryGenerator composes domain, style, audience, speaker perspective, purpose, language, materials, and checkable cross-slide requirements. Accepted SFT trajectories retain planning, delegation, rendered inspection, and refinement. THREAD-Bench then evaluates process and artifact evidence plus long-range probes; PresentBench, SlidesGen-Bench, and DECKBench cover complementary generation and editing behavior. Our contributions are:

1. a Skill-defined presentation-authoring lifecycle spanning research, planning, grouped production, two-level inspection, post-generation revision, and multi-format delivery;
2. a dependency-aligned multi-Agent responsibility topology in which slide groups jointly own related pages, preserve inter-group parallelism, and support selective re-execution by revision scope; and
3. THREAD-Bench, which evaluates long-horizon authoring from both process and artifact evidence, together with systematic general-generation and multi-turn-revision evaluation of MURAL.

2 Related Work

2.1 Presentation Generation across the Deck Lifecycle

Document-to-presentation systems study content selection, reorganization, and slide structure (Bandyopadhyay et al., 2024; Maheshwari et al., 2024). Agent systems extend the lifecycle: PPTAgent plans from references and an outline, PreGenie reviews code and renders (Zheng et al., 2025; Xu et al., 2025), and SlideBot and DeepPresenter connect specialized roles, shared state, and rendered reflection (Xie et al., 2026; Zheng et al., 2026). These are foundations, not our isolated novelty claims. MURAL instead connects them through a shared blueprint and derives production and replay boundaries from cross-slide dependencies.

2.2 Long-Horizon State and Multi-Agent Granularity

Long-horizon difficulty concerns both history growth and state allocation. Context Folding shows that accumulated histories can impair earlier-information use (Sun et al., 2025); LongHorizon-Harness externalizes verified task state and alternates a manager, fresh-context executor, and read-only auditor (Ma et al., 2026); controlled multi-Agent studies further show that decomposability, information sharing, and validation placement affect outcomes (Kim et al., 2025). These works motivate explicit state and independent verification, but not a particular presentation-production boundary. MURAL instead asks which distant slide relations require a common owner: groups persist as joint production, inspection, and revision units, while whole-deck Review closes relations that cross groups.

2.3 Agentic Data, Refinement, and Revision

Agent tuning increasingly preserves decomposition, tool calls, and interaction rather than only final answers (Zeng et al., 2024; Chen et al., 2024); iterative feedback can improve generation (Madaan et al., 2023). MURAL’s agentic SFT also retains render inspection, diagnosis, patches, and rerendering. Whether generation-time refinement transfers to later editing remains an experiment, tested by removing refinement traces while controlling the base model and data.

2.4 Presentation Generation and Editing Benchmarks

PresentBench checks material-grounded requirements, SlidesGen-Bench evaluates Content, Aesthetics, and artifact Editability, and DECKBench includes generation and multi-turn editing (Chen et al., 2026; Yang et al., 2026; Jang et al., 2026). PPTArena and PPT-Eval cover compound, cross-slide, or PowerPoint-native edits (Ofengenden et al., 2025; Gandhi et al., 2026); LH-Bench evaluates process and output with Skill-grounded rubrics (Gupta & Chandwani, 2026). THREAD-Bench claims none of these firsts. It instead joins process evidence, artifact quality, and case-specific long-range relations for from-scratch deck authoring, reporting closure by dependency span. SlidesGen-Bench Editability remains an artifact property, not multi-turn Agent revision.

3 Long-Horizon Agentic Data Synthesis

3.1 Query Specifications, Not Just Topics

QueryGenerator constructs a specification rather than a bare topic. It combines domain, style, language, target length, materials, audience, purpose, and speaker perspective, then adds 2–4 checkable cross-slide requirements: for example, a reused definition, a persistent visual encoding, or an opening question answered later. Long-range state is thus part of the task before a trajectory exists.

The inventory contains 1,000 unique specifications (850 Chinese; 150 English), drawn from 22 top-level domain files, 306 unique leaves, and 43 styles. These counts describe the query inventory, not accepted training yield or real-world frequency. We distinguish N_{query} , $N_{rollout}$, $N_{complete}$, and $N_{accepted}$: one query may yield multiple rollouts, and completion does not imply acceptance.

3.2 Dual-Model Rollout and Trajectory Acceptance

Planned DeepSeek-V4-Flash and Gemini-3.5-Flash rollouts execute the same MURAL Authoring Skill. Besides a final deck, they expose material and knowledge briefs, the shared blueprint, group/page briefs, assets, HTML, contact sheets, Review findings, and patches. Endpoints, allocation, sampling, and the final mixture remain release-freeze fields.

Deterministic gates check required artifacts, page coverage, builds, and rendering; rubric checks cover instruction satisfaction, grounding, and visible defects; deduplication limits near-duplicates. Failed or unverifiable trajectories remain in the accounting but are not relabeled as accepted data. The frozen release will report all four counts and rejection reasons.

Accepted examples retain planning, delegation, tool use, render inspection, diagnosis, patching, and rerendering. We call this agentic SFT because supervision targets an executable authoring trajectory, not only final HTML. The data are not a dedicated editing corpus, but their refinement turns contain operations used in later revision. E5 and a matched-data ablation test whether that supervision transfers to multi-turn editing without harming untouched slides; until then, refinement is a hypothesis, not an explanation of editing ability.

4 MURAL

4.1 Task and Long-Horizon State

Long decks require facts, terminology, narrative setup, and visual encodings to survive distant pages, while each slide still needs role-specific wording and composition. MURAL is designed neither to maximize Agent count nor parallelism, but to align execution responsibility with the scope of those dependencies. It records deck decisions as persistent state, assigns related pages to accountable production units, and verifies cross-unit relations after assembly; slide count alone is not the objective.

Given brief q , attachments $\mathcal{M}$, language ℓ , and length N , the system produces deck $D = (s_1, \dots, s_N)$, notes $U = (u_1, \dots, u_N)$, renders, and trace Π . Shared deck state G stores facts and sources, terminology, narrative relationships, visual tokens, and delivery requirements. We represent a long-horizon dependency as $g_k = (a_k, T_k, \Phi_k)$, where source a_k establishes a decision, targets T_k must consume or respond to it, and Φ_k tests the relation. Deck-level dependency closure is

$$\text{DC}(D) = \frac{1}{K} \sum_{k=1}^K \mathbf{1} \left[\bigwedge_{t \in T_k} \Phi_k(s_{a_k}, s_t) = 1 \right].$$

This separates local quality from decision persistence. Production groups form a complete, non-overlapping slide partition. If $\{a_k\} \cup T_k$ lies in one group, one Group Agent owns the dependency; otherwise, the shared blueprint and whole-deck Review connect it. MURAL avoids one all-deck group, which would recreate an unbounded trajectory.

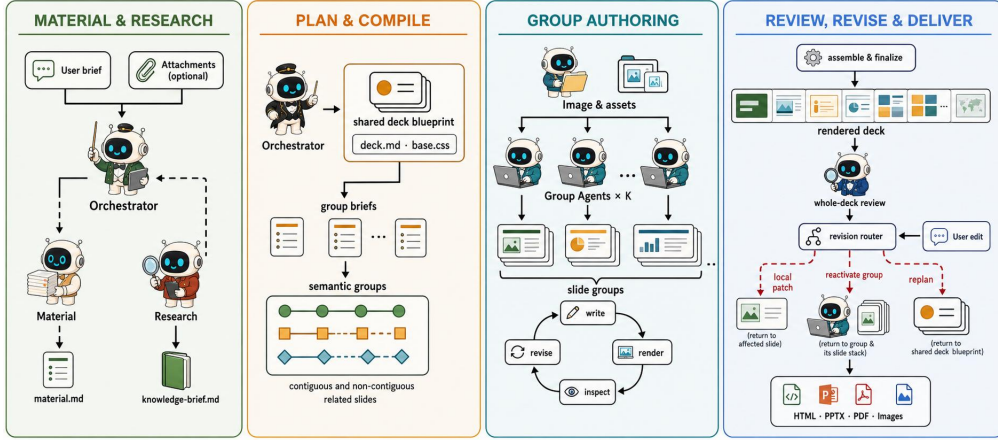


Figure 2: The full authoring and revision lifecycle in MURAL. Material and Research ground the inputs, after which the Orchestrator externalizes deck-level decisions as a shared blueprint and group briefs. A centralized Image stage prepares assets before Group Agents run parallel render-inspect-revise loops. Deterministic assembly restores the deck view for whole-deck Review. The same revision router handles review findings and human edits by patching a page, replaying a group, or returning structural changes—including page insertion or deletion—to deck planning before exporting HTML, PPTX, PDF, and images.

4.2 Skill-Defined Lifecycle

The MURAL Authoring Skill defines stage triggers, role inputs, artifacts, quality gates, and revision routes. Material organizes optional attachments; Research grounds knowledge; the Orchestrator maintains state and delegates; Image resolves assets; Group Agents author related slides; and Review restores deck-wide judgment. The runtime supplies workspaces, concurrency, and traces, while decisions remain in explicit artifacts rather than dialogue history.

Research records claims, evidence, and source locations from the brief and optional materials. The Orchestrator resolves audience, storyline, terminology, design language, page responsibilities, and asset strategy. A centralized Image stage records selections in assets/catalog.md; versioned deck.py fetch-images handles retrieval and validation, so the model has no custom fetch_image tool.

Figure 2 summarizes the lifecycle from grounding and planning through grouped authoring, whole-deck Review, scope-aware revision, and delivery.

4.3 Shared Blueprint and Dependency-Aligned Responsibility Topology

The Orchestrator externalizes evidence, narrative, terminology, design, page map, assets, and groups in plan/deck.md; plan/slide_NN.md projects page-relevant state. Validation compiles theme tokens, base.css, skeletons, and notes. Group Agents share one factual and design source of truth without inheriting peer histories.

A slide group is a responsibility unit between one page and the deck. Rather than splitting by stage, index, or fixed window, the Orchestrator uses narrative purpose, setup-response relations, design DNA, assets, and explicit dependencies. The implementation groups cover with closing, all section dividers together, and content by continuous argument and design affinity. Groups may be contiguous, non-adjacent, or singleton; common ownership does not imply one template.

Thus, each Agent maintains a bounded group trajectory, within-group relations share an author and inspector, and groups become revision replay boundaries. Inter-group parallelism follows from these boundaries; it is not the primary claim.

4.4 Group-Parallel Authoring and Two-Level Closure

Each Group Agent receives the blueprint, briefs, skeletons, and assets. In one bounded trajectory, it authors its pages, renders a group contact sheet, consolidates defects, revises, and rechecks. Groups coordinate through the blueprint and persisted artifacts rather than frequent negotiation; disjoint source ownership allows safe parallelism.

After groups converge, deterministic finalization checks continuity, assembles authoritative HTML, and renders a whole-deck contact sheet. Review checks factual conventions, terminology, setup and response, visual semantics, and rhythm. Findings return to the smallest accountable unit before re-finalization and deck-level reinspection.

4.5 Revision Routing and Delivery

Post-generation requests reuse this state. `local_revision` patches one page; `group_revision` reactivates its Group Agent; `evidence_revision` refreshes Research or Image; and `global_revision` updates the blueprint and groups for structural changes. Uncertain requests escalate one level. This limits planned replay; revision quality and preservation are tested empirically.

Accepted state is exported from structured HTML/CSS/SVG to HTML, PNG, PDF, and PPTX. These adapters share one authoring source but have different fidelity and editability limits, which are reported separately from generation quality.

5 THREAD-Bench

5.1 Missing Observables and Case Construction

Slide 3 may map blue to confirmed and gray to pending verification; Slide 18 may reverse them. Each looks valid alone, but their mismatch makes the deck undeliverable. Facts, terms, questions, and requirements can drift similarly. THREAD-Bench encodes source-to-target relations, localizing reuse by distance.

The current design targets 50 expert-constructed deck tasks organized into five 10-case suites: visual system, narrative logic, cross-slide state, grounded synthesis, and constraint coordination. Equal suite size is a coverage device, not a claim that real tasks or capabilities are disjoint. Cases vary deck length, language, material availability, and constraint density; they are deliberate stress tests rather than samples from production traffic.

Each case stores the user brief, optional frozen materials, expected delivery format, verifiable requirements, long-range dependency probes, and slide-localizable evidence rules. The exact slide total, language/material distribution, core/challenge split, and derived requirement/dependency counts are release-manifest fields. They remain provisional in this working draft and will be reported only after the canonical case index and materials are frozen.

5.2 Requirements, Dependencies, and Horizon

Each case separates verifiable must requirements from non-vetoing should preferences. A must enters the hard gate, so subjective quality cannot compensate for a wrong slide count, a missing mandatory fact, or a prohibited design. Deck-level must-requirement satisfaction is one co-primary endpoint; release counts for both classes will be generated from the frozen manifest rather than copied from a draft inventory.

Dependencies track state reuse. A probe establishes a fact, term, question–response relation, visual encoding, or task decision at a source and names one or more targets; closure requires every named target to preserve or consume it correctly. For source and target indices s and t , span is

$$d = t - s.$$

Target judgments localize failure; group closure requires all targets in the probe to succeed. The release report will bin targets at 1–4, 5–9, 10–14, and 15+ slides after recomputing the distribution from the frozen case index. Case-first aggregation prevents dependency-rich tasks from receiving disproportionate weight.

Natural medium, long, and ultra slices vary topic, density, and requirements and are descriptive only. E2 uses content-matched 10/20/30/40-slide anchor variants, fixing core facts, dependency types, and budget rules while changing source/target layout. Only these support stronger horizon interpretations; non-orthogonal count and span are reported jointly.

5.3 Metrics, Reporting Conventions, and Reproducibility Boundaries

THREAD-Bench reports no weighted aggregate score. Co-primary endpoints are deck-level must satisfaction and dependency-group closure. The stricter `deployable_as_is` requires completion and every applicable hard gate and must. Diagnostics report span-binned accuracy; factual/numerical, terminology/state, narrative, and visual consistency; and eight separate 1–5 axes. Completed, failed, timed-out, and unverifiable runs remain in all 50 cases’ denominator. Quality axes are reported over all cases and conditional on completion; the latter is not primary.

Requirement/dependency judgments retain category and slide-localizable evidence. In E7, at least two humans measure inter-human and human–Judge agreement on an independent calibration set; Judge outputs cannot support primary conclusions before calibration. Benchmark and material manifests store case/schema and material hashes; the E0 manifest freezes Judge prompt, rubric, model, and aggregation code. Contamination audits cover accessible sources only; unavailable training corpora preclude a zero-contamination claim. The current task design is specification-heavy; natural briefs and hidden acceptance criteria remain future work.

6 Experiments

6.1 Research Questions and Common Protocol

RQ1 tests general quality, requirements, and dependency closure; RQ2 span and horizon; RQ3 consistency failures; RQ4 generic decomposition versus dependency-aligned ownership, shared state, and two-level inspection; RQ5 revision, preservation, and replay scope; and RQ6 Judge reliability. Runtime, tokens, and cost are secondary.

Comparisons fix backbone, temperature, inputs, renderer, budget, retry limit, failure rules, and 3 seeds per case. Seeds are preregistered and aggregated within case; the case is the highest analysis unit. Paired bootstrap resamples cases then seeds, reports two-sided 95% CIs, and applies Holm correction to co-primary comparisons. Missing artifacts, failures, and timeouts remain in the denominator; failed outputs score 1 where applicable, with N/A axes and scored coverage separate. Uncontrollable closed products are references only.

6.2 E0: Reproducibility Freeze

E0 freezes systems, 50 cases, E2 anchors, independent E7 calibration data, code/Skill revision, renderer, backbone, inference settings, budgets, seeds, failure rules, and Judge/aggregation versions. Separate hashes cover the benchmark, materials, “THREAD-Bench evaluation spec,” and “MURAL system-comparison protocol”; mismatches halt execution. Concurrency, Clean Skill selection, and thinking/effort are logged only for reproduction. Commit 168791d... is the current candidate Skill snapshot: its runtime, special-slide, and role/runtime checks do not imply that the full stack is frozen.

6.3 E1: General Quality and Main Complete-Deck Comparison

E1a follows PresentBench and SlidesGen-Bench protocols for content, grounding, aesthetics, and artifact structure; external systems appear separately with model, budget, renderer, and format differences disclosed.

E1b compares all 50 THREAD-Bench cases under (1) one full-lifecycle Agent; (2) stage-specialized Agents without dependency-aware groups; (3) full-context per-slide workers sharing one blueprint; and (4) MURAL group ownership. Backbone, inputs, renderer, tools, and total budget are matched; exceptions are disclosed. This isolates generic decomposition from boundary alignment.

After deterministic checks, a frozen system-blinded Judge evaluates THREAD-Bench. Table 2 reports PresentBench and SlidesGen-Bench; Table 3 reports must-requirement satisfaction, dependency closure, deployable-as-is, span accuracy, four consistency categories, eight quality axes, completion, slide-count compliance, and scored coverage. Conclusions cover only the frozen release, backbone, budget, and benchmark.

Main comparison pending: TBD-E1.

6.4 E2: Dependency Span and Deck Horizon

E2a reports case-balanced target accuracy over 1–4, 5–9, 10–14, and 15+ slide spans, stratified by dependency type, position, and system when samples permit. Content differences limit these to complete-task associations.

E2b derives matched 10/20/30/40-slide families from at least 6 anchors, crossing short/long spans where feasible. Facts, dependency types, content blocks, visual systems, and budget rules stay fixed; layout and length-maintaining content change. Analysis covers system, pages, span, and preregistered interactions using family-cluster bootstrap and within-family randomization. Non-orthogonal families report only joint horizon–span effects; Figure 3 shows closure, target accuracy, consistency, and paired intervals.

Span/matched-family estimates pending: TBD-E2.

6.5 E3: Consistency and Failure Types

E3 preregisters factual/numerical drift, terminology/state conflict, narrative setup/handoff/closure failure, visual/deck-rule drift, requirement omission, unsupported grounding, and rendering/artifact failure. Targets may carry multiple labels, but “any failure” and each type remain separate.

Analysis reports case-level rates, first occurrence, span, early/late targets, and co-occurrence. Figure 4 pairs representative source/target slides with localized Judge evidence, locating failures by horizon without causal module attribution.

Failure-type analysis pending: TBD-E3.

6.6 E4: Responsibility Topology and Lifecycle Mechanisms

E4 separates Agent count from responsibility boundaries. Its 50-case panel tests (1) per-slide ownership with the blueprint and whole-deck Review; (2) size-matched fixed-adjacent groups; (3) no group-joint inspection, using page checks instead; and (4) local-only Review. Size-matched random groups are preregistered diagnostics on a fixed 10-case subset.

A fixed-subset lifecycle panel tests Generic Skill, No shared blueprint/group brief, and No Review by replacing role-specific guidance, removing the deck source of truth, or skipping post-assembly inspection. Each variant changes one factor; a comparison that jointly changes topology, tools, context, and inspection budget cannot be labeled simply “with/without multiple Agents” or “with/without a Skill.”

A matched-data panel compares final-artifact SFT, agentic trajectories with inspect–diagnose–patch turns removed, and full accepted trajectories while holding query identities, accepted-example count, optimization tokens, base model, and training recipe fixed. E1 measures generation effects and E5 revision transfer; editing gains without preservation do not count as successful transfer.

Table 5 reports requirements, overall and within-/cross-group closure, span-conditioned accuracy, four consistency categories, deployable-as-is, and completion. Inspection variants

report repair/regression rates; regressions use previously correct, verifiable opportunities. Within-group gains and cross-group losses remain separate.

Topology, lifecycle, and data-supervision ablations pending: TBD-E4.

6.7 E5: Multi-Turn Revision and Selective Replay

E5 uses DECKBench to compare scope-aware routing, local-only routing, and per-turn full-deck replay from the same initial deck and requests. It reports per-turn/sequence success, instruction following, consistency, untouched-slide preservation, new regressions, replay scope, and relaunched stages. HTML conversion failures and non-editable elements remain separate; replay scope verifies selective execution but cannot replace revision quality.

Multi-turn revision pending: TBD-E5.

6.8 E6–E7: System Accounting and Judge Calibration

E6. E1/E2/E4/E5 log time, critical path, calls, tokens, cost, peak context/overlap, render/view counts, worker duration, and three concurrency values. Workspace snapshots distinguish model write_file/patch from terminal/deck.py mutations. These are secondary reproducibility context, not abstract-level evidence (TBD-E6).

E7. Before E1, at least two annotators label requirements, dependencies, consistency, and quality on frozen data disjoint from E1; resolution locks the Judge, evidence format, and thresholds. A stratified post-E1 audit estimates error without retuning. Binary labels use nominal α/κ , 1–5 axes use ordinal α or weighted κ ; low-agreement dimensions remain exploratory (TBD-E7).

Legacy-Release Trace Audit. Pilot B reuses old traces without new model calls. Their plan/pages.json, research_01, and old scaffold cannot validate the current workflow; they only show that persisted traces expose worker intervals, file reads, and artifact failures. Appendix statistics are not current-release quality or long-horizon evidence.

7 Discussion and Limitations

Group ownership makes responsibility explicit, but Agent count alone creates no benefit: outcomes depend on whether the Orchestrator finds the right dependency boundaries. Poor partitions can split relations that need a common owner or create oversized trajectories. Missed or evolving cross-group dependencies still rely on whole-deck Review; tightly coupled animations and composite figures may not fit non-overlapping ownership.

The 1,000 data records are query specifications, not accepted trajectories; rollout yield, filtering effects, and training benefit remain unmeasured. Synthetic constraints may overrepresent easy-to-state requirements and underspecify tacit audience expectations. Ablations must separate query diversity, accepted-trajectory quality, and refinement supervision.

THREAD-Bench remains an expert stress test with limited grounding, incomplete matched families, and an uncalibrated Judge. Its exact inventory is provisional until the release manifest is frozen, while HTML and its export adapters limit generalization. E1–E7 must therefore report repairs with regressions and judge effectiveness through dependency closure rather than runtime or cost.

8 Conclusion

MURAL treats presentations as persistent authoring lifecycles. Rather than merely creating more subtasks, it aligns production and replay boundaries with deck dependencies. A shared blueprint, slide groups, two-level review, and scope-aware revision preserve decisions across pages and edits; THREAD-Bench tests whether those dependencies close over longer horizons.

References

- Sambaran Bandyopadhyay, Himanshu Maheshwari, Anandhavelu Natarajan, and Apoorv Saxena. Enhancing presentation slide generation by LLMs with a multi-staged end-to-end approach. In Proceedings of the 17th International Natural Language Generation Conference, pp. 222–229, Tokyo, Japan, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.inlg-main.18. URL <https://aclanthology.org/2024.inlg-main.18/>.
- Xin-Sheng Chen, Jiayu Zhu, Pei-lin Li, Hanzheng Wang, Shuojin Yang, and Meng-Hao Guo. Presentbench: A fine-grained rubric-based benchmark for slide generation. arXiv preprint arXiv:2603.07244, 2026.
- Zehui Chen, Kuikun Liu, Qiuchen Wang, Wenwei Zhang, Jiangning Liu, Dahua Lin, Kai Chen, and Feng Zhao. Agent-flan: Designing data and methods of effective agent tuning for large language models. In Findings of the Association for Computational Linguistics: ACL 2024, pp. 9354–9366, 2024.
- Apurva Gandhi, Vishwas Suryanarayanan, Raja Hasnain Anwar, Firoz Shaik, Shubhang De-sai, Thong Q. Nguyen, Muhammad Taqi Raza, Vishal Chowdhary, and Graham Neubig. PPT-Eval: A benchmark for computer-use agents on PowerPoint tasks. In Forty-third International Conference on Machine Learning, 2026.
- Ishan Gupta and Abhishek Chandwani. LH-Bench: Skill-grounded evaluation of long-horizon agents on subjective enterprise tasks, 2026. URL <https://arxiv.org/abs/2603.22744>. arXiv:2603.22744v2.
- Daesik Jang, Morgan Lindsay Heisler, Linzi Xing, Yifei Li, Edward Wang, Ying Xiong, Yong Zhang, and Zhenan Fan. Deckbench: Benchmarking multi-agent frameworks for academic slide generation and editing. arXiv preprint arXiv:2602.13318, 2026.
- Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. Towards a science of scaling agent systems. arXiv preprint arXiv:2512.08296, 2025. URL <https://arxiv.org/abs/2512.08296>.
- Ziyu Ma, Hailang Huang, Shun Zou, Yong Wang, Shidong Yang, Yiming Hu, Fei Wei, and Xiangxiang Chu. LongHorizon-Harness: Advancing long-horizon agents for real-world tasks, 2026. URL <https://arxiv.org/abs/2608.01964>. arXiv:2608.01964v1.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wier-effe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In Advances in Neural Information Processing Systems (NeurIPS), 2023.
- Himanshu Maheshwari, Sambaran Bandyopadhyay, Aparna Garimella, and Anandhavelu Natarajan. Presentations are not always linear! GNN meets LLM for text document-to-presentation transformation with attribution. In Findings of the Association for Computational Linguistics: EMNLP 2024, pp. 15948–15962, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.936. URL <https://aclanthology.org/2024.findings-emnlp.936/>.
- Michael Ofengenden, Yunze Man, Ziqi Pang, and Yu-Xiong Wang. PPTArena: A benchmark for agentic PowerPoint editing. arXiv preprint arXiv:2512.03042, 2025. URL <https://arxiv.org/abs/2512.03042>.
- Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon LLM agent via context-folding, 2025.
- Eric Xie, Danielle Waterfield, Michael Kennedy, and Aidong Zhang. Slidebot: A multi-agent framework for generating informative, reliable, multi-modal presentations. In AAAI Conference on Artificial Intelligence (AAAI), 2026.

Xiaojie Xu, Xinli Xu, Sirui Chen, Haoyu Chen, Fan Zhang, and Ying-Cong Chen. PreGenie: An agentic framework for high-quality visual presentation generation. In Findings of the Association for Computational Linguistics: EMNLP 2025, pp. 3045–3063, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-emnlp.165. URL <https://aclanthology.org/2025.findings-emnlp.165/>.

Yunqiao Yang, Wenbo Li, Houxing Ren, Zimu Lu, Ke Wang, Zhiyuan Huang, Zhuofan Zong, Mingjie Zhan, and Hongsheng Li. SlidesGen-Bench: Evaluating slides generation via computational and quantitative metrics. arXiv preprint arXiv:2601.09487, 2026. URL <https://arxiv.org/abs/2601.09487>.

Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. Agenttuning: Enabling generalized agent abilities for llms. In Findings of the Association for Computational Linguistics: ACL 2024, pp. 3053–3077, 2024.

Hao Zheng, Xinyan Guan, Hao Kong, Wenkai Zhang, Jia Zheng, Weixiang Zhou, Hongyu Lin, Yaojie Lu, Xianpei Han, and Le Sun. Pptagent: Generating and evaluating presentations beyond text-to-slides. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 14402–14418, 2025.

Hao Zheng, Guozhao Mo, Xinru Yan, Qianhao Yuan, Wenkai Zhang, Xuanang Chen, Yaojie Lu, Hongyu Lin, Xianpei Han, and Le Sun. Deeppresenter: Environment-grounded reflection for agentic presentation generation. arXiv preprint arXiv:2602.22839, 2026.

A Draft Appendix

Reproducibility ledger. Commit 168791d20585d3d7a667540215736c4b639a2458 is retained as the current candidate Skill snapshot together with critical-file hashes, recorded regression checks, and the special-page smoke test. It is not yet the complete experimental freeze. The ledger also records Clean Skill selection, thinking/effort, three distinct concurrency controls, and the catalog-driven fetch-images script. Formal baseline configurations, renderer and model settings, canonical benchmark manifests, Judge prompts, human-annotation protocol, and full per-case results will be added only after E0 locks the complete stack.

Secondary operational accounting. Wall-clock time, critical-path duration, calls, tokens, estimated cost, context overlap, renders, and the three concurrency controls are retained for reproducibility and deployment accounting. They are not definitions of long horizon and do not replace deck-level requirement satisfaction, dependency closure, or the four consistency families in the main analysis.

Pilot interpretation. The working draft does not use provisional benchmark inventory counts as evidence. Pilot B applies the audit code to persisted traces from a legacy release. It is not a controlled comparison of generation quality and does not validate the current grouped workflow.

A.1 Pilot B: Legacy-Release Trace Audit

Pilot B reuses two completed runs and incurs no new model calls. Both contain plan/pages.json, the research_01 shard, and the legacy scaffold contract, all of which predate the current Markdown-plan candidate. The grouped workflow instead has the Orchestrator write plan/deck.md and plan/slide_NN.md, after which scaffold-from-plans deterministically generates a lightweight HTML skeleton and speaker notes. Measured from the first page-generator event to trace completion, the 3-page run has wall/worker-sum times of 407.272/759.235 seconds, a parallel compression ratio of 1.864 $\times$, peak concurrency of 3, and 3/3 page generators consistent with the read instructions in force at the time. The corresponding figures for the 6-page run are 369.030/1049.931 seconds, 2.845 $\times$, 6, and 5/6. In the latter run, slide_05 reads slide_03.html; the persisted tool log records no direct writes to peer/shared source files. The audit cannot observe provider-hidden context or file changes

induced through terminal actions, and it does not establish the current release’s execution boundaries, quality, or relative savings.

The two runs record 73/151 calls and 1.93M/6.44M input tokens, respectively. The 6-page task explicitly disallows the default cream-white palette, yet still uses #ede2cc/#e2d3b3 in base.css after Review completes. This single case cannot estimate a failure rate, but it shows that completion, rendering, and boundary compliance cannot substitute for the E1 requirement judge. Boundary statements about the current release come from static code inspection; actual execution compliance must still be tested on new traces after E0.

Pilot artifacts. Machine-readable legacy-trace statistics are stored in pilot_b_trace_audit.json and pilot_b_worker_intervals.csv. The bounded source/contact-sheet check is documented in pilot_b_output_sanity.md; source hashes are retained there.